Arthur Augusto Lara Matos

Prática 5: Consumindo API em JS
Desenvolvimento de Aplicações Web

Belo Horizonte, Brasil

16/04/2026

1 Introdução

Este projeto consiste no desenvolvimento de uma aplicação para a visualização de notícias em carrossel, estruturada por meio de `JavaScript` e da biblioteca `p5.js`. A construção do sistema exige a aplicação da função `fetch()` para o consumo e tratamento dinâmico de dados provenientes de uma *API* do G1. Dessa forma, o algoritmo gerencia temporizadores independentes para orquestrar um carrossel, consolidando conceitos fundamentais de manipulação de texto, lógica assíncrona e controle temporal.

1.1 Conceito

O conceito desta atividade reside na integração entre dados externos e a renderização gráfica. Para atingir esse objetivo, o projeto explora a programação assíncrona como mecanismo fundamental para buscar e tratar respostas em formato `JSON` provenientes de uma *API*. O cerne arquitetural da aplicação baseia-se na teoria de *Finite State Machines*, que gerencia de forma determinística as transições entre as etapas de carregamento de rede e os modos de reprodução ou pausa do carrossel. Somado a isso, o sistema exige o domínio lógico do controle temporal, operando temporizadores independentes para coordenar a rotação visual das notícias e a atualização periódica do pacote de dados.

1.2 Objetivos

O objetivo desta atividade é capacitar o desenvolvimento de aplicações *front-end* dinâmicas, integrando a renderização gráfica do `p5.js` com o consumo de dados externos. Especificamente, visando aprofundar o domínio da programação assíncrona mediante o uso da função `fetch()` para capturar, tratar e higienizar informações da *API*. Além disso, a atividade busca consolidar a aplicação prática do padrão de Máquinas de Estados. Essa estruturação técnica permite orquestrar, de forma simultânea e independente, a rotação do carrossel de notícias .

2 Desenvolvimento

2.1 Estratégia

A estratégia de desenvolvimento adotada fundamenta-se na separação modular entre a lógica de manipulação de dados e a camada de renderização gráfica. Inicialmente, a aplicação emprega uma abordagem assíncrona estruturada com blocos `try/catch` para consumir a *API* externa, garantindo que o tratamento do `JSON` e a higienização textual não bloqueiem o laço de repetição principal da biblioteca `p5.js`. Para orquestrar o fluxo da interface, a estratégia apoia-se em uma arquitetura orientada a estados (*State-Driven Architecture*), utilizando dicionários literais para blindar e alternar as condições de disponibilidade dos dados (carregando, pronto

ou erro) e o modo de interação do usuário (reprodução ou pausa). O motor de execução desse sistema baseia-se na checagem contínua de deltas temporais através da função `millis()`, uma abordagem não obstrutiva que permite calcular de forma concorrente a rotação visual das notícias a cada dez segundos e a renovação dos dados em segundo plano a cada minuto.

2.2 Passos

Síntese dos passos utilizados na criação do código:

- Estruturação da Máquina de Estados.
- Consumo Assíncrono da *API*.
- Higienização e tratamento de texto.
- Controle temporal independente.
- Renderização do carrossel e indicadores.
- Implementação de interatividade e pausa.

3 Resultados

3.1 Subseções do código final

Este trecho de código configura as variáveis globais e estabelece a arquitetura de controle. Inicialmente, armazena-se o endereço do servidor externo na constante `API`. Em seguida, consolida-se o padrão de Máquina de Estados por meio dos objetos literais `PLAYER_STATE` e `DATA_STATE`, que atuam como enumeradores semânticos para gerenciar os ciclos de interação do usuário (Play/Pause) e as fases de carregamento da rede. Por fim, são declaradas as variáveis mutáveis de gerenciamento estrutural, englobando o vetor para armazenamento do `JSON`, o ponteiro de navegação do carrossel (`indiceAtual`) e os registradores temporais baseados em milissegundos.

```
const API =
    "https://api.rss2json.com/v1/api.json?rss_url=https://g1.globo.com/dynamo/bras

const PLAYER_STATE = {
  PLAYING: "playing",
  PAUSED: "paused"
};

const DATA_STATE = {
  LOADING: "loading",
  READY: "ready",
```

```
  ERROR: "error"
};

let estadoPlayer = PLAYER_STATE.PLAYING;
let estadoDados = DATA_STATE.LOADING;

let noticias = [];
let indiceAtual = 0;

//Tempo
let ultimaTroca = 0;
let ultimaConsulta = 0;
let tempoRestante = 10000; //10 segundos = 10000ms

let linkNoticia;
```

Estabelece as configurações iniciais do ambiente visual e define o laço de renderização da aplicação. Na função de inicialização (`setup`), o escopo gráfico é dimensionado e um elemento interativo de hiperlink `HTML` é instanciado nativamente pelo `p5.js`. Esse elemento recebe uma série de propriedades diretas de estilização no `CSS`, sendo posicionado e mantido oculto até que a primeira chamada à *API* retorne os dados necessários. Em paralelo, a função principal (`draw`) atua como o motor do algoritmo, limpando o plano de fundo a cada *frame* e delegando as responsabilidades de controle de estado e construção gráfica para as funções auxiliares.

```
function setup() {
  createCanvas(800, 500);

  //cria o botao de link uma unica vez e o oculta ate ter dados
  linkNoticia = createA("#", "Abrir noticia", "_blank");
  linkNoticia.style("background-color", "#8a0500ff");
  linkNoticia.style("color", "white");
  linkNoticia.style("padding", "10px 20px");
  linkNoticia.style("text-decoration", "none"); //tira a barrinha de
      link
  linkNoticia.style("border-radius", "5px");
  linkNoticia.style("font-family", "Arial");
  linkNoticia.style("font-weight", "bold"); //negrito
  linkNoticia.position(width / 2 - 60, height / 2 + 80);
  linkNoticia.hide();

  //primeira chamada da API
  pegarNoticias();
}

function draw() {
  background(230); //cinza
```

```

  atualizarLogica();
  desenharInterface();
}
```

Encapsula a rotina assíncrona responsável pelo consumo da *API* de notícias e pelo controle fundamental da máquina de estados. A função inicia alterando a condição do sistema para o modo de carregamento e ocultando temporariamente o *link* da interface. Em seguida, utiliza uma estrutura de tratamento de exceções (`try/catch`) combinada com o comando `await` para realizar a requisição HTTP e aguardar a conversão da resposta para o formato `JSON`. Em caso de sucesso, o algoritmo armazena o vetor de informações, reinicia os três contadores temporais baseados na função `millis()` e altera o estado para pronto (`READY`). Caso ocorra alguma falha de conexão, a exceção é devidamente capturada e registrada, transferindo o sistema para a condição de erro, o que impede a quebra abrupta da aplicação.

```
// LOGICA DE DADOS E ESTADOS

async function pegarNoticias() {
  estadoDados = DATA_STATE.LOADING;
  linkNoticia.hide();

  try {
    const resposta = await fetch(API);
    if (!resposta.ok) throw new Error("Erro HTTP");

    const dados = await resposta.json();
    noticias = dados.items;
    indiceAtual = 0;

    //reseta os temporizadores
    ultimaConsulta = millis();
    ultimaTroca = millis();
    tempoRestante = 10000;

    estadoDados = DATA_STATE.READY;
    atualizarLink(); //atualiza o href do elemento DOM

  } catch (erro) {
    console.error("Erro ao buscar dados:", erro);
    estadoDados = DATA_STATE.ERROR;
  }
}
```

Encapsula o motor de controle temporal e a lógica de navegação do carrossel circular. A rotina principal gerencia dois temporizadores concorrentes com base nos deltas da função `millis()`, assegurando a renovação assíncrona dos dados da *API* a cada sessenta segundos

e o avanço visual da interface a cada dez segundos, condicionado estritamente à validação prévia da máquina de estados (garantindo que o sistema não esteja pausado e que os dados já estejam carregados). Ao acionar a transição, o algoritmo incrementa o índice de leitura do *array* e aplica uma verificação de limite para retornar a zero ao atingir o final da lista, redefinindo simultaneamente a base do cronômetro. Por fim, a atualização sincroniza-se com a manipulação da interface de marcação, injetando dinamicamente a *URL* da nova manchete no atributo de hiperligação (`href`) para o redirecionamento externo do usuário.

```
function atualizarLogica() {
  //controle de atualizacao da API (60 segundos)
  if (millis() - ultimaConsulta > 60000) {
    pegarNoticias();
  }

  //controle do carrossel (10 segundos)
  if (estadoDados === DATA_STATE.READY && estadoPlayer ===
     PLAYER_STATE.PLAYING) {
    if (millis() - ultimaTroca > 10000) {
      avancarNoticia();
    }
  }
}

function avancarNoticia() {
  indiceAtual++;
  //carrossel circular
  if (indiceAtual >= noticias.length) {
    indiceAtual = 0;
  }

  //reinicia o tempo para a nova noticia
  ultimaTroca = millis();
  tempoRestante = 10000;
  atualizarLink();
}

function atualizarLink() {
  if (noticias.length > 0) {
    linkNoticia.attribute("href", noticias[indiceAtual].link);
    linkNoticia.show();
  }
}
```

Ilustra a aplicação prática da Máquina de Estados na camada de renderização visual. A função atua como um controlador condicional que avalia o estado atual do processamento de dados (`estadoDados`). Caso o sistema esteja em fase de requisição ou enfrente erros, a

interface exibe mensagens textuais de *feedback* na tela para orientar o usuário. Contudo, quando as informações da *API* estão plenamente carregadas e disponíveis (`READY`), a rotina altera o fluxo de execução e delega a renderização para funções modulares especializadas, encarregadas de desenhar o conteúdo da notícia ativa, o indicador de progresso temporal e os botões interativos.

```
// DESENHO DA INTERFACE

function desenharInterface() {
  if (estadoDados === DATA_STATE.LOADING) {
    fill(50);
    textAlign(CENTER, CENTER);
    textSize(24);
    text("Carregando noticias", width / 2, height / 2);

  }
  else if (estadoDados === DATA_STATE.ERROR) {
    fill(200, 0, 0);
    textAlign(CENTER, CENTER);
    textSize(24);
    text("Erro ao carregar notucias. Tentando novamente", width / 2,
       height / 2);

  }
  else if (estadoDados === DATA_STATE.READY) {
    desenharNoticia(noticias[indiceAtual]);
    desenharIndicadorTempo();
    desenharBotaoPlayer();
  }
}
```

O trecho de código encapsula a rotina de renderização visual de uma notícia individual na interface gráfica da aplicação. Inicialmente, a função constrói um contêiner centralizado na tela, estruturado como um *card* claro, que atua como plano de fundo para organizar o conteúdo. Em seguida, o algoritmo estabelece uma hierarquia tipográfica clara: o título é renderizado em destaque após ser submetido a uma função auxiliar de truncamento, prevenindo a quebra inadequada do leiaute. Por fim, a descrição da notícia passa por um processo essencial de higienização (`limparHTML`) para remover eventuais formatações estruturais oriundas da *API*.

```
function desenharNoticia(noticia) {
  //card branco central
  rectMode(CENTER);
  fill(255);
  noStroke();
  rect(width / 2, height / 2 - 20, 700, 300, 15);

  //texto
```

```
  fill(30);
  textAlign(CENTER, TOP);

  //titulo da noticia
  textSize(22);
  textStyle(BOLD);
  let titulo = resumir(noticia.title, 90);
  text(titulo, width / 2, height / 2 - 100, 600, 100);

  //descricao da Noticia
  textSize(16);
  textStyle(NORMAL);
  fill(80);
  let descLimpa = limparHTML(noticia.description);
  let descResumida = resumir(descLimpa, 200);
  text(descResumida, width / 2, height / 2 - 30, 600, 150);
}
```

Este trecho de código é responsável por calcular e renderizar a barra de progresso visual que indica o tempo restante para a transição automática das notícias. Inicialmente, a função avalia a máquina de estados: caso o reprodutor esteja ativo, calcula-se dinamicamente os milissegundos faltantes subtraindo o tempo decorrido do ciclo total de dez segundos; caso esteja pausado, o valor é congelado em sua última marcação, com uma trava de segurança impedindo que eventuais atrasos de *frame* gerem valores negativos. Na etapa de renderização, o algoritmo desenha um trilho de fundo estático e emprega a função matemática `map()` para interpolar o tempo numérico restante em uma proporção física de *pixels*. Esse cálculo determina a largura exata da barra colorida sobreposta, fazendo com que ela encolha de forma linear e perfeitamente sincronizada.

```
function desenharIndicadorTempo() {
  //calcula dinamicamente o tempo que falta
  let tempoCalculado;
  if (estadoPlayer === PLAYER_STATE.PLAYING) {
    tempoCalculado = 10000 - (millis() - ultimaTroca);
  }
  else {
    tempoCalculado = tempoRestante; //se pausado, congela no tempo
        restante
  }

  //nao exibe valores negativos por atrasos de frame
  if (tempoCalculado < 0) tempoCalculado = 0;

  //barra de fundo
  let larguraMax = 700;
  rectMode(CORNER);
```

```
  fill(220);
  rect(width / 2 - 350, height / 2 + 150, larguraMax, 6, 3);

  //barra de progresso
  let larguraProgresso = map(tempoCalculado, 10000, 0, 0, larguraMax);
  fill(196, 23, 12);
  rect(width / 2 - 350, height / 2 + 150, larguraProgresso, 6, 3);
}
```

É responsável pela renderização e interatividade visual do botão de controle do reprodutor. A função define a área geométrica do botão na parte inferior central da tela e utiliza o cálculo matemático de distância (`dist`) entre a posição atual do cursor e o centro do círculo para criar um efeito reativo de mudança de cor, simulando um *hover*. Em seguida, o algoritmo consulta a máquina de estados da aplicação (`PLAYER_STATE`) para determinar o ícone apropriado: caso o sistema esteja em execução, o símbolo textual de pausa é sobreposto ao círculo; caso contrário, a função constrói geometricamente o triângulo representativo de *Play*, garantindo uma interface dinâmica e um *feedback* visual coerente com a lógica temporal do sistema.

```
function desenharBotaoPlayer() {
  let btnX = width / 2;
  let btnY = height - 40;
  let raio = 25;

  //cor do botao (Muda se o mouse passar por cima - hover)
  if (dist(mouseX, mouseY, btnX, btnY) < raio) {
    fill(70);
  }
  else {
    fill(40);
  }
  circle(btnX, btnY, raio * 2);

  //Play ou Pause
  fill(255);
  textAlign(CENTER, CENTER);
  textSize(18);

  if (estadoPlayer === PLAYER_STATE.PLAYING) {
    text("      ", btnX, btnY); //icone de pause
  } else {
    //desenha o triangulo de Play
    //ajustado para centralizar
    triangle(btnX - 4, btnY - 8, btnX - 4, btnY + 8, btnX + 8, btnY);
  }
}
```

Gerencia a interatividade do usuário e o refinamento dos dados textuais a serem exibidos. A função `mousePressed()` atua como um ouvinte de eventos, utilizando cálculos de distância geométrica bidimensional (`dist()`) para identificar cliques exatos sobre o botão circular da interface. Ao detectar a interação, a rotina alterna os estados da máquina entre reprodução e pausa, executando um cálculo compensatório sofisticado com a função `millis()`: ela armazena o tempo exato restante do ciclo e, ao retomar, reajusta a variável de referência, criando a ilusão de que o cronômetro interno foi perfeitamente congelado. Paralelamente, o algoritmo implementa duas rotinas de tratamento de *strings*: a função `resumir()`, que trunca parágrafos longos baseando-se no último espaço em branco para não quebrar palavras pela metade, e a rotina `limparHTML()`, que instancia um elemento virtual no *DOM* do navegador para converter e extrair apenas o texto puro da *API*.

```
// INTERACAO E OUTROS

function mousePressed() {
  let btnX = width / 2;
  let btnY = height - 40;
  let raio = 25;

  //verifica se clicou em cima do botao Play/Pause
  if (dist(mouseX, mouseY, btnX, btnY) < raio && estadoDados ===
      DATA_STATE.READY) {

    if (estadoPlayer === PLAYER_STATE.PLAYING) {
      estadoPlayer = PLAYER_STATE.PAUSED;

      //salva o tempo exato que restava
      tempoRestante = 10000 - (millis() - ultimaTroca);

    } else {
      estadoPlayer = PLAYER_STATE.PLAYING;

      //engana o millis(), empurrando a ultimaTroca
      //parece que o tempo nao passou enquanto esteve pausado
      ultimaTroca = millis() - (10000 - tempoRestante);
    }
  }
}

function resumir(texto, limite) {
  if (texto.length <= limite) return texto;
  let corte = texto.substring(0, limite);
  let ultimoEspaco = corte.lastIndexOf(" ");
  return corte.substring(0, ultimoEspaco) + "...";
}

```

```
function limparHTML(html) {
  let div = document.createElement("div");
  div.innerHTML = html;
  return div.textContent || div.innerText || "";
}
```

3.2 Resultados finais

O projeto foi totalmente bem-sucedido: o desenvolvimento de um motor gráfico otimizado para um carrossel utilizando `JavaScript` e `p5.js` cumpriu integralmente os requisitos propostos pelo professor. A abordagem estrutural baseada em Programação Orientada a Objetos e no uso de uma *API* provou-se bem eficaz. Apesar dos desafios inerentes à implementação da lógica vetorial, especialmente no ajuste da gravidade da subclasse e no monitoramento constante das instâncias para deleção ativa, nenhum deles comprometeu a estabilidade final.

A seguir, apresenta-se uma visualização da execução do programa:

Figura 1 – Resultado do carrossel de notícias.

4 Conclusão

Os resultados foram ótimos, cumprindo com sucesso o propósito pedagógico e prático da atividade. Mesmo com alguns desafios de lógica e codificação durante a execução, foi possível superá-los com pesquisas em fóruns e o documento do professor, sem comprometer o

andamento do projeto. No fim das contas, a atividade foi excelente para aprimorar a lógica de `JavaScript`.